# Lecture 1 - Introduction to Machine Learning

## Table of Contents



---

## 1. Why Machine Learning?

Modern computing systems increasingly operate in environments where:

- Data is abundant ("Big Data"[1])
- Explicit rules are difficult to specify
- Adaptation is required

Traditional programming follows the pattern[2]:

> Rules (written by programmer) + Data → Output

Example: program after coding

```
if (email contains "win money") -> spam
else -> not spam
```

Machine learning reverses this relationship[3]:

> Data + Outputs → Rules (model) Rules + New Data → Output

Example: model after training[4]

```
if (learned_score(email) > threshold) -> spam
else -> not spam
```

Applications:

- Email spam detection
- Credit card fraud detection
- Image classification

Remember:

- Traditionally, rules are explicitly written.
- In machine learning, rules are learned from data.

---

# 2. What is Machine Learning?

- Machine learning can be defined as:

  > Giving computers the ability to learn without being explicitly programmed.

- Or more specifically as:

  > Learning a function from data

- This involves four basic components:

  1. **Features (inputs)** — measurable quantities (email sender)
  2. **Target (output)** — what we want to predict (if email is spam or ham)
  3. **Model** — a mathematical function mapping inputs to outputs (Naive Bayes)
  4. **Loss function** — a measure of prediction error (Cross entropy)

- What's the goal?

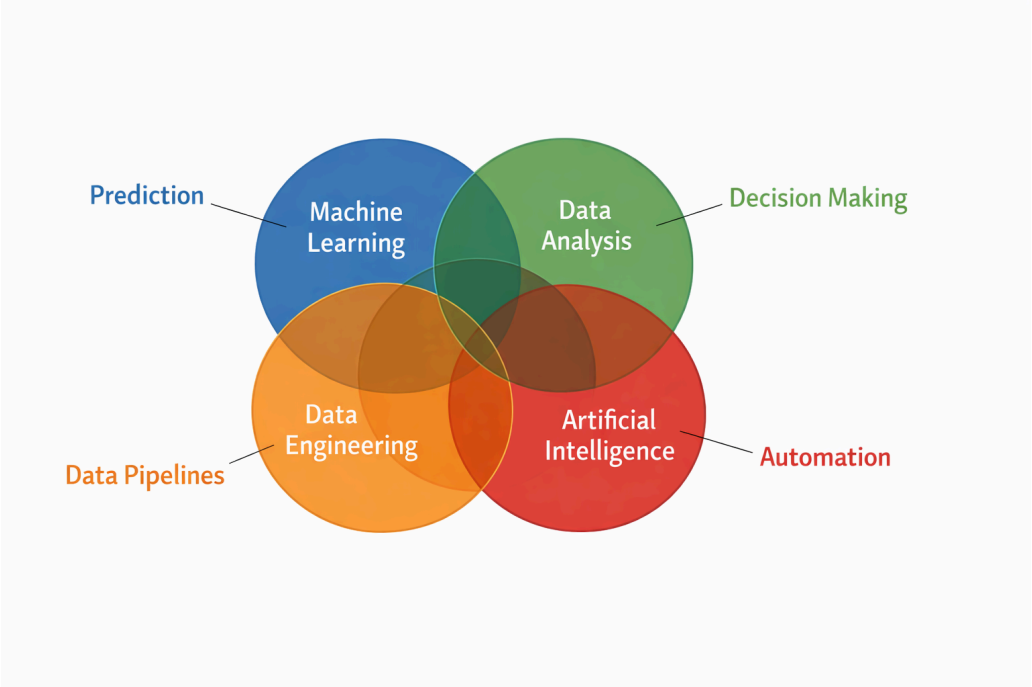
  > The goal is not to memorize the data, but to perform well on new, unseen data.

- What's a useful model?

  > A model that performs well on training data but poorly on new data is not useful.

ML is not, but overlaps with:

- Data processing
- Data analysis
- Artificial Intelligence




---

## 3. Machine learning applications and techniques

- Can you think of useful applications for predictive machine learning?

| APPLICATION | TECHNIQUE |
|---|---|
| Analyzing product images on a production line to classify them | Image classification with CNNs or transformers |
| Detecting tumors in brain scans | Semantic image segmentation with CNNs or transformers |
| Automatically classify news articles | NLP text classification with CNNs or transformers |
| Flag offensive comments on forums | NLP text classification |
| Summarizing long documents | NLP text |
| Creating a chatbot or assistant | NLP + QA modules, transformers |
| Forecasting company revenue from KPIs | Regression: linear, polynomial, SVM, random forest |
| Making apps react to voice commands | Speech recognition: CNNs, transformers, or RNNs |
| Detecting credit card fraud | Anomaly detection: isolation forest, GMM, autoencoders |
| Segmenting clients by purchases | Clustering: k-means, DBSCAN Grouping data based on distance |

| APPLICATION | TECHNIQUE |
|---|---|
| Visualizing high-dimensional data | Dimensionality reduction |
| Recommending products from past purchases | Recommender systems using ANN (trained on purchase patterns) |
| Building game-playing bots | Reinforcement learning (maximize a reward function) |

---

# 4. Simple Example: Spam filter

A spam filter using traditional programming:

1. Examine what spam looks like. Identify patterns.
2. Write a detection algorithm for spam patterns.
3. Flag emails as spam if spam patterns are detected.
4. Test program until the algorithm is good enough.

A spam filter using machine learning:

1. Label a bunch of emails as spam.
2. Train the machine to identify spam. Create a model.
3. Test the model on new (unseen) data.
4. Improve model's ability to detect spam until it is good enough.

Whan a new spam pattern appears:

- The human algorithm has to be updated with a new rule.
- The ML model is updated by retraining on the new data.

How is the machine learning approach better?

- The ML model re-training can be automated when new data appear
- The program is much shorter, more accurate, and easier to maintain
- The ML approach scales to Big Data easily (many emails, much spam)
- Models can be used for data mining to improve understanding
- ML can be applied in fluctuating environments with big data

What's the bottleneck of machine learning?

- Data may be of **low quality** (noise, mislabeling)
- Example: legitimate emails are incorrectly labeled as spam (or vice versa), causing the model to learn wrong patterns.
- Data may be **insufficient** (during training)
- Example: only a small number of spam emails are available, so the model cannot learn reliable indicators of spam.
- Data may be **non-representative** (biased)
- Example: training data comes mostly from one source (e.g., corporate emails), so the model performs poorly on personal or international emails.
- Data may be **imbalanced**
- Example: 95% of emails are non-spam and only 5% are spam, leading the model to favor predicting “not spam.”
- Data may be **missing important features**
- Example: the dataset does not include sender reputation or email metadata, which are strong indicators of spam.
- Data may be **including irrelevant features**
- Example: features like email ID numbers or timestamps unrelated to spam are included, adding noise and confusing the model.

---

# 5. Types of Machine Learning

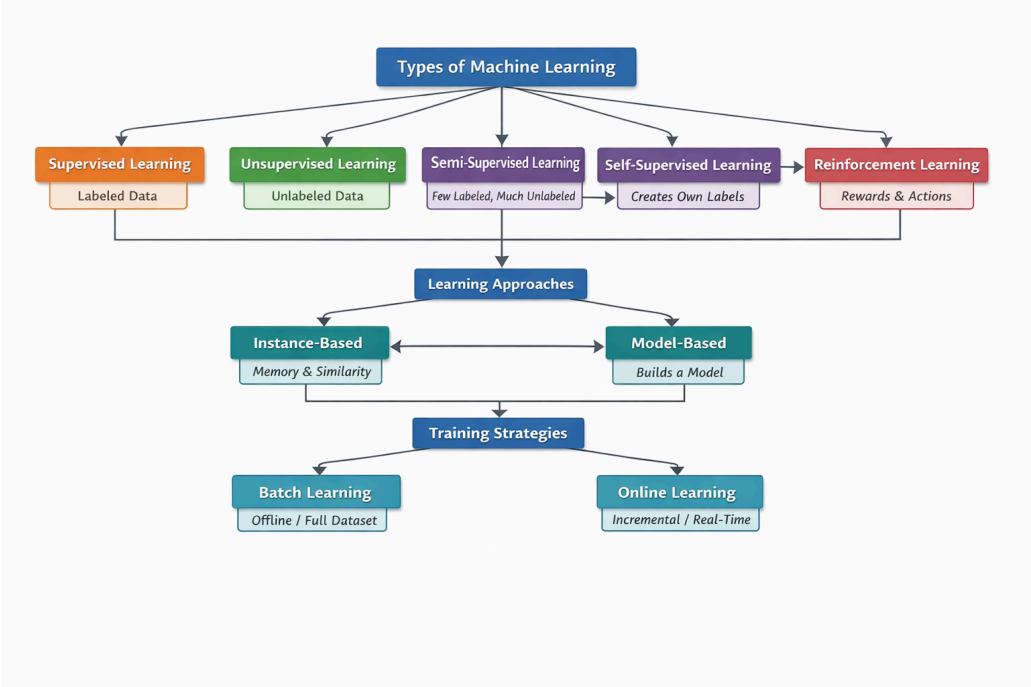



Figure 1: ML types and paradigms overview

We distinguish three main types of machine learning:

1. Supervised learning (labeled data sets)
2. Unsupervised learning (unlabeled data sets)

Between these two, there exist many different algorithms.

## 5.1. Supervised Learning

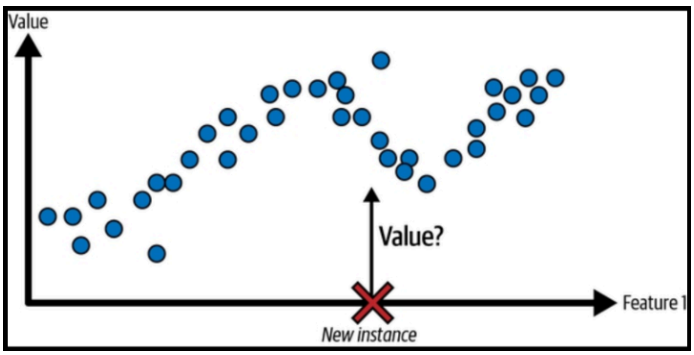



Figure 2: Regression - predict new value

- Data: includes both inputs and correct outputs
- Goal: learn mapping from inputs to outputs
- Price: data to learn from must be labeled

Examples:

- Predicting house prices (regression)
- Classifying emails as spam or not spam (Bayes)

- Diagnosing illness from symptoms (classification)

## 5.2. Unsupervised Learning

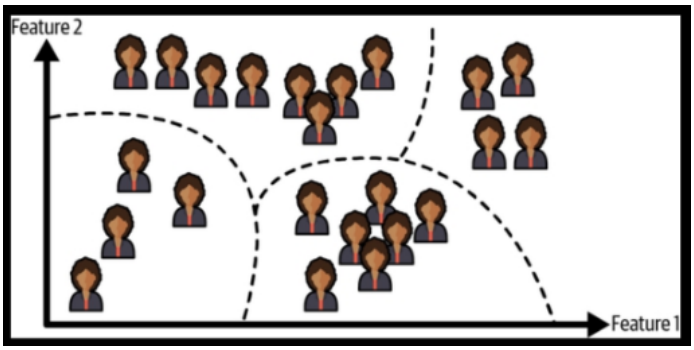
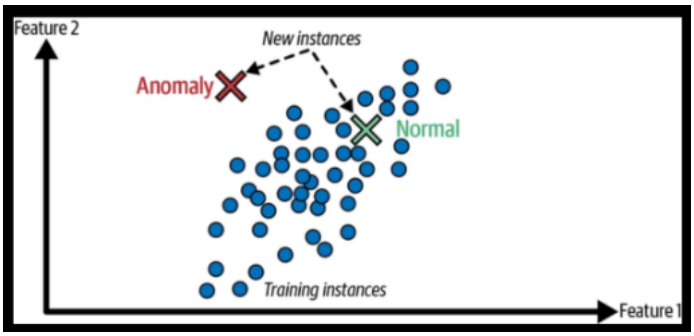



Figure 3: Clustering - group by features

- Data: includes inputs only
- Goal: discover structure in the data
- Price: model may be mislead more easily (no ground truth)

Examples:

- Clustering customers (group by category)
- Dimensionality reduction (make problem simpler)
- Detecting anomalies (unusual transactions)




## 5.3. Semi-supervised Learning

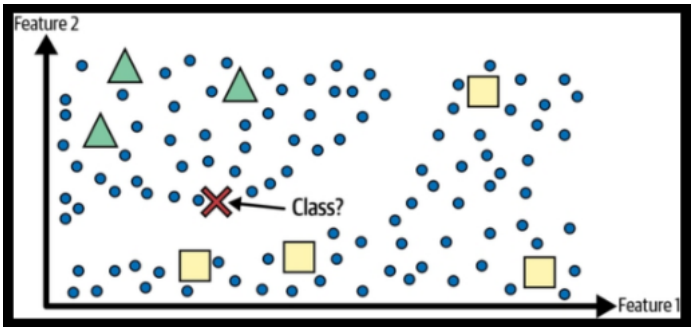


Figure 4: Unlabeled data help classify a new instance

- Data: small amount labeled, large amount unlabeled[5]
- Goal: leverage structure in unlabeled data to improve predictions
- Price: assumptions about data structure may be wrong

Examples:

- Classifying emails when only few are labeled spam/ham
- Image classification w/few labeled, many unlabeled images
- Speech recognition w/limited transcribed audio, large raw datasets

## 5.4. Self-supervised Learning

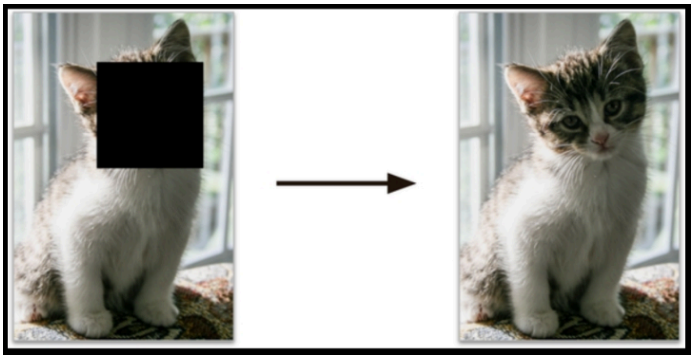


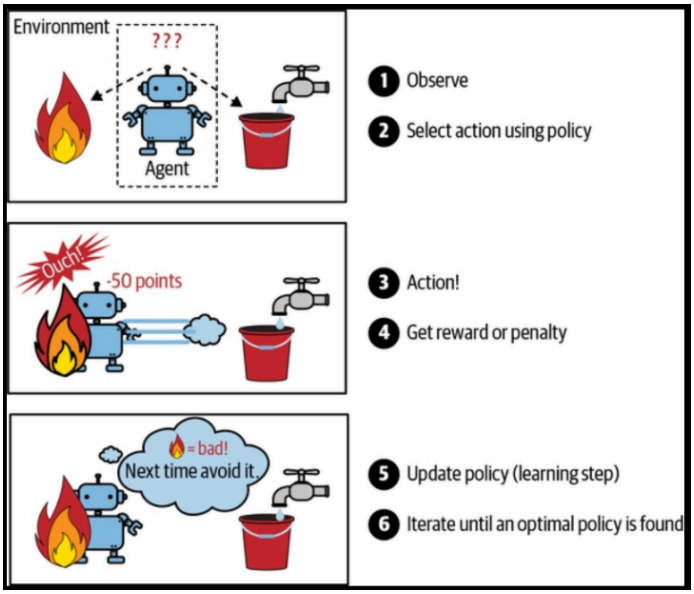
Figure 5: Masked input (left) and target image (right)

- Data: uses unlabeled data, creates its own labels from data[6]
- Goal: learn useful representations by solving auxiliary tasks
- Price: learned task may not align perfectly with final objective

Examples:

- Predicting missing words in a sentence (language models)
- Predicting next word/token in text (transformers)
- Filling in masked parts of an image (vision models)

## 5.5. Reinforcement learning

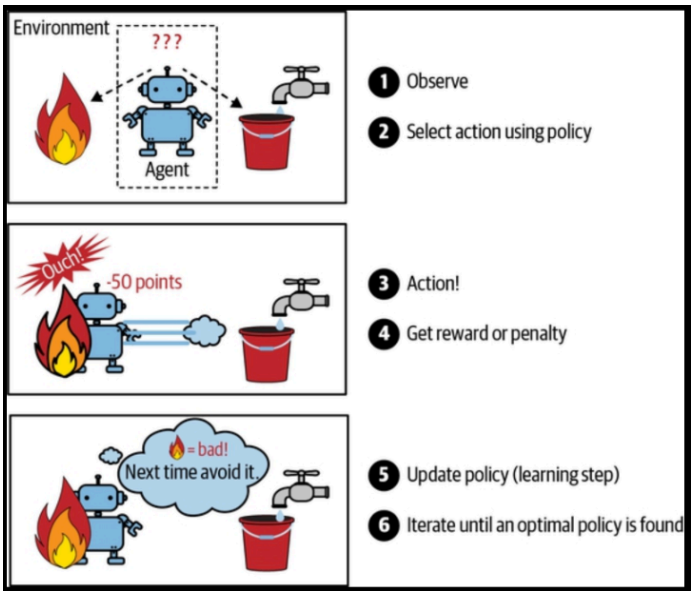



- Data: generated through interaction with environment
- Goal: learn policy that maximizes long-term reward
- Price: exploration costly, learning unstable, slow

Examples:

- Training a game-playing agent (chess)
- Robotics control (learning to walk)
- Optimizing recommendations through user interaction

## 5.6. Batch vs. Online learning




#+begin$_{quote}$

| Batch Learning | Online Learning |
|---|---|
| Data: full dataset upfront | Data: sequential / stream |
| Goal: train on all data | Goal: update continuously |
| Price: costly retraining | Price: noise sensitivity |
| Examples: nightly spam | Example: real-time spam siltering |
| training with regression | with live recommendation updates |

## 5.7. Online learning

The model is updated incrementally, one data point (or small batch) at a time, as new data arrives.

If the data does not fit in main memory, only part of the data is loaded, runs a training step then loads the next part (out-of-core learning, done incrementally, in practice often off-line).

Important parameter: **learning rate** = how fast should the model adapt to changing data. High rate adapts quickly but forgets old data too. Low rate leads to more inertia (slower learning) and less sensitivity to noise in new data or to outliers.

## 5.8. Instance-based vs. Model-based learning

This relates to the ability of the models to **generalize** = making good predictions for examples it has never seen before. Two approaches dominate:

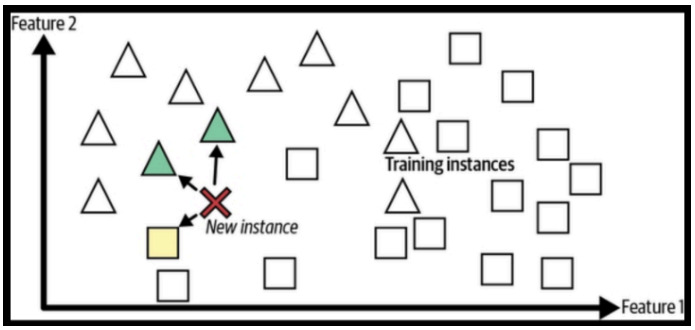



Figure 6: New instance is a triangle because majority of similar instances are

For instance-based learning, the system learns the examples by heart, then generalizes to new cases using a similarity meausre to compare them to the learned examples.

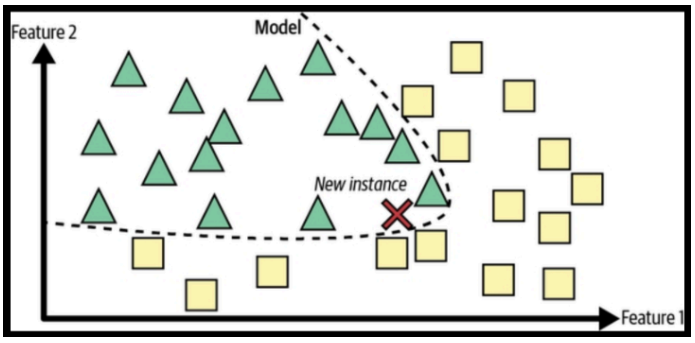



Figure 7: Build a model to make predictions of new data

# 6. Extended Example: Does money make people happy? (Difficult)

Gerón uses a rather difficult example. You can find it with code in a Colab notebook - you can run the code cells in the notebook.

1. Download data: Better Life Index from OECD/World Bank
2. Join tables and sort by GDP per capita ("money")
3. Visualize the data
4. Select a linear model (because of the visualization)
5. Train the model (use Python library)
6. Make a prediction
7. Pick a different model

Instead, we will use two simpler examples - a decision tree and a linear regression (trendline fit)

---

# 7. Extended Example: How can you diagnose illness? (Simple)

Consider diagnosing an illness based on symptoms.

- Input (features): symptoms (e.g., fever, cough)
- Output (target): diagnosis (e.g., flu, cold)
- Model: a decision rule (or tree)

The model attempts to approximate the relationship:

> diagnosis ≈ f(symptoms)

**Process:**

1. Build a **classification** model from **known** data instances
2. Test model to **classify** newly presented **unknown** data instances
3. Translate model into algorithmic **production** rules

## 7.1. Building a model from training data

- Dataset: hypothetical training data for a disease diagnosis

| Patient ID# | Sore Throat | Fever | Swollen Glands | Congestion | Headache | Diagnosis |
|---|---|---|---|---|---|---|
| 1 | Yes | Yes | Yes | Yes | Yes | Strep throat |
| 2 | No | No | No | Yes | Yes | Allergy |
| 3 | Yes | Yes | No | Yes | No | Cold |
| 4 | Yes | No | Yes | No | No | Strep throat |
| 5 | No | Yes | No | Yes | No | Cold |
| 6 | No | No | No | Yes | No | Allergy |
| 7 | No | No | Yes | No | No | Strep throat |
| 8 | Yes | No | No | Yes | Yes | Allergy |
| 9 | No | Yes | No | Yes | Yes | Cold |
| 10 | Yes | Yes | No | Yes | Yes | Cold |

Figure 8: Hypothetical disease diagnosis training data (Source: Roiger, 2021)

- Observation: Patient 1 has a sore throat, fever, swollen glands, is congested and has a headache. He was diagnosed with strep throat.
- A *decision tree* can be used to generalize a set of input instances as shown and transform it into rules.
- To generalize, we must make assumptions about the relative importance of attributes and their relationship.

- For example:
    - If a patient has swollen glands, the diagnosis is strep throat

- If a patient does not have swollen glands and a fever, it's a cold
- If a patient does not have swollen glands nor a fever, it's allergy

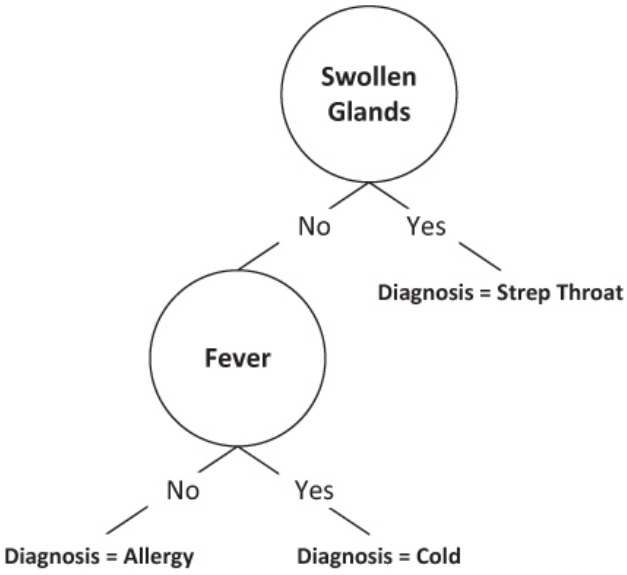



Figure 9: Decision tree for disease diagnosis (Source: Roiger, 2021)

- Notice that the attributes *sore throat*, *congestion* and *headache* do not enter our diagnostic prediction.

## 7.2. Testing the model on unknown instances

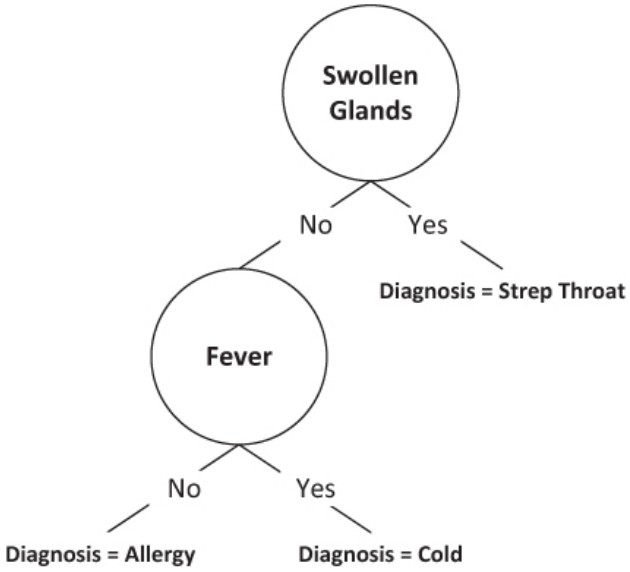



Figure 10: Decision tree for disease diagnosis

- Moving on to a new data set with unknown classification, i.e. no diagnosis
- Use the decision tree to classify the first two instances:

| Patient ID | Sore Throat | Fever | Swollen Glands | Congestion | Headache | Diagnosis |
|---|---|---|---|---|---|---|
| 11 | No | No | Yes | Yes | Yes | ? |
| 12 | Yes | Yes | No | No | Yes | ? |
| 13 | No | No | No | No | Yes | ? |

Figure 11: Test data for disease diagnosis model (source: Roiger, 2021)

- Patient 11 has swollen glands but no fever => strep throat
- Patient 12 has no swollen glands but fever => cold

## 7.3. Translate model into production rules

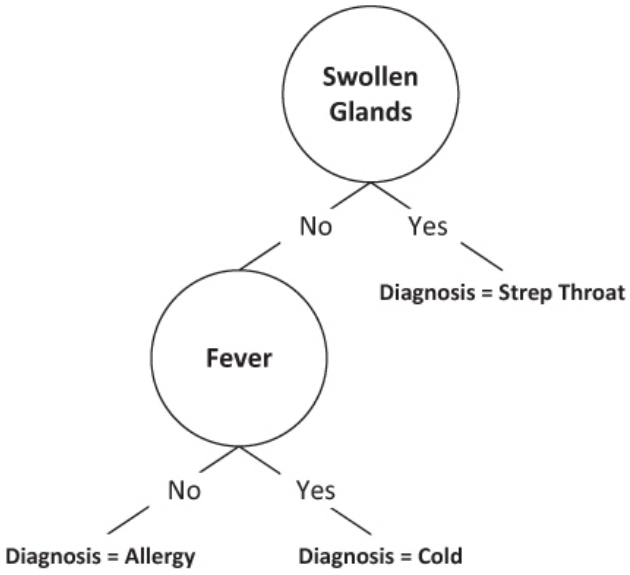



Figure 12: Decision tree for disease diagnosis (source: Roiger, 2021)

- General form of a *production rule* looks like pseudocode[7]:

```
IF antecedent condition
THEN consequent conditions
```

- The three *production rules* for the decision tree:

```
IF swollen glands = YES
THEN diagnosis = strep throat

IF swollen glands = No & Fever = Yes
THEN diagnosis = cold

IF swollen glands = No & Fever = No
THEN diagnosis = allergy
```

- Testing the rules on patient 13 yields: diagnosis = allergy

| Patient ID | Sore Throat | Fever | Swollen Glands | Congestion | Headache | Diagnosis |
|---|---|---|---|---|---|---|
| 11 | No | No | Yes | Yes | Yes | ? |
| 12 | Yes | Yes | No | No | Yes | ? |
| 13 | No | No | No | No | Yes | ? |

Figure 13: Test data for disease diagnosis model (source: Roiger, 2021)

### 7.4. PRACTICE Testing and improving a classification model

This practice exercise consists of four parts:

1. Test the model on new patients using the diagnostic table
2. Modify the production rules by adding a new attribute
3. Draw the updated decision tree model

Time = 15 minutes

Handout (submit solution for points)

---

## 8. PRACTICE AI with Python

We're going to do our ML coding in Python using AI (Gemini) in Google Colaboratory.

1. Why AI?
2. Why Python?
3. Why AI Python?
4. Demo: How chatbots interact with Python
5. Home assignment: Build a 'guess a number' game.

See separate practice file $AI_{Python.org}$

---

## 9. The Machine Learning Workflow

A typical workflow consists of the following steps:

1. Define the problem
2. Collect data
3. Prepare data
4. Choose a model
5. Train the model
6. Evaluate the model
7. Use the model for prediction

This process is iterative. At any stage, it may be necessary to revisit earlier steps.

### 9.1. Common Issues

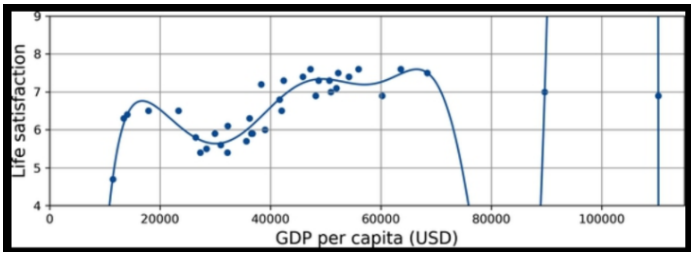



Figure 14: Overfitting the training data

- Overfitting: model learns noise instead of pattern
- Underfitting: model is too simple to capture structure

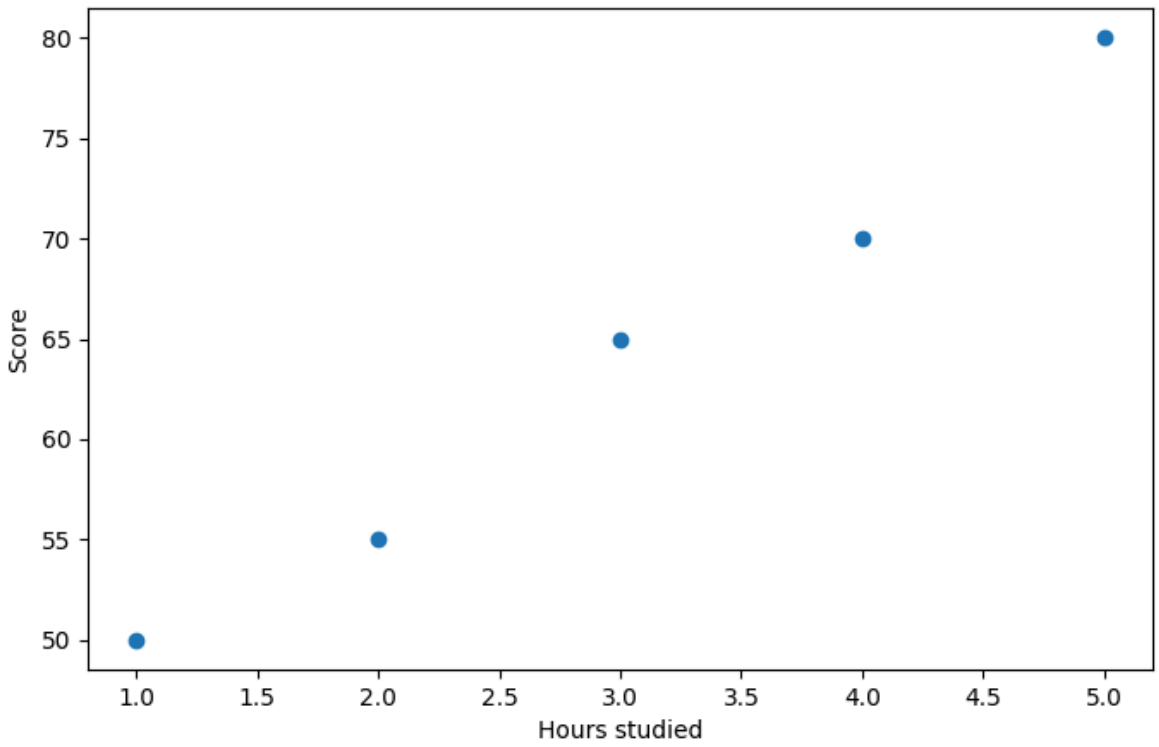
---

## 10. PRACTICE Fitting a linear model to data (Simple)

Open the colab notebook and code along: [tinyurl.com/linear-regression-practice](tinyurl.com/linear-regression-practice)

- Data visualization without modeling

```
import matplotlib.pyplot as plt
import numpy as np
hours = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
score = np.array([50, 55, 65, 70, 80])

plt.scatter(hours,score)
plt.xlabel("Hours studied")
plt.ylabel("Score")
plt.savefig("hours_score.png")
```




- The following example fits a linear model to data.

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Data
hours = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
score = np.array([50, 55, 65, 70, 80])

# Model
model = LinearRegression()
model.fit(hours, score)

# Parameters
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
```

```
Slope: 7.500000000000001
Intercept: 41.5
```

- Code explanation:
  1. We import the numerical math library, `numpy`, and alias it as `np`, and we import the `LinearRegression` class from the machine learning library `scikit-learn`.
  2. We create two arrays, `hours` and `score`.
  3. Many ML libraries expect the input feature matrix `X` in 2D form. `reshape` changes the `shape` of the array without changing the data:

```
print(np.array([1, 2, 3, 4, 5]).shape) # 1D array with shape (5,)
print(hours.shape)                     # 2D array: 5 rows, 1 column
```

```
(5,)
(5, 1)
```

4. An empty `model` instance is created by `LinearRegression()`.
5. `model.fit(hours, score)` trains the model: `X` is the input (`hours`), and `y` is the output (`score`). The method solves an optimization problem: it finds the values of $\beta_0$ and $\beta_1$ that minimize the sum of squared residuals (the sum of error squared):

$$\sum_{i=1}^{n} (y_i - (\beta_0 + \beta_1 x_i))^2$$

6. After `.fit()`, `model.coef_` ($\beta_1$) and `model.intercept_` ($\beta_0$) are available. They define the learned function: `prediction = intercept + slope * x`.
7. Numeric example: `hours[2] = 3` and `score[2] = 65`. For x=3, the model predicts

$$\hat{y} = 41.5 + 7.5 \cdot 3 = 64.0$$

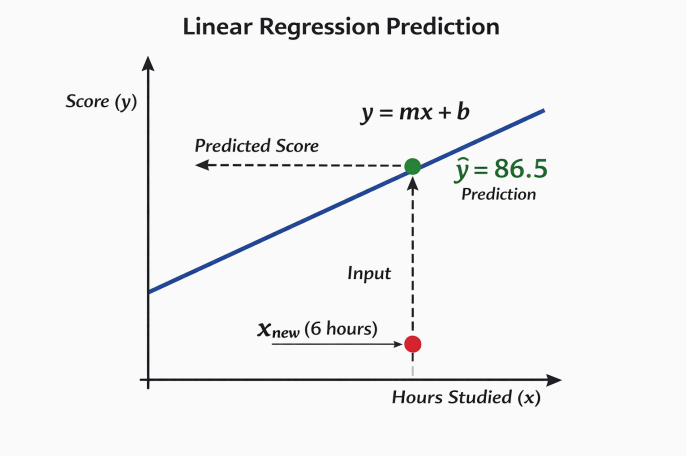
so the residual is

$$y - \hat{y} = 65 - 64 = 1.0$$

There are five such data equations but only two unknown parameters, so this is an **overdetermined system**.
8. The prediction answers the question: If I study `x` hours, what score does the model predict?
9. Each data point acts like a **constraint**. Since there are more constraints than unknowns, they usually cannot all be satisfied exactly, so we find the line that is closest overall in the least-squares sense.
10. There are many different possible loss functions, not just least squares. Which is best depends on the problem distribution.

---

# 11. Making Predictions




- The trained model can be used to make predictions:

```
print(model.predict([[6]]))
```

```
[86.5]
```

- This returns the predicted score for 6 hours of study.
- Code explanation:
    1. After training, the model has learned parameters $\beta_0$ (intercept) and $\beta_1$ (slope).

2. The method `model.predict(X_new)` uses these parameters to compute predicted outputs for new input data.
3. Internally, it applies the learned function:

$$\hat{y} = \beta_0 + \beta_1 x$$

   to each input value.
4. The argument to `predict` must have the same **shape** as the training input `X`, i.e., a 2D array of shape `(n_samples, n_features)`.
5. Since we trained with one feature (`hours`), the model expects input of shape `(?, 1)`.
6. The input `[[6]]` is therefore:
   - one sample (outer list)
   - one feature (inner list)

   so its shape is `(1, 1)`.
7. Using `[6]` instead would create a 1D array with shape `(1,)`, which is not accepted by `scikit-learn` for prediction.
8. The call `model.predict([[6]])` returns an array:

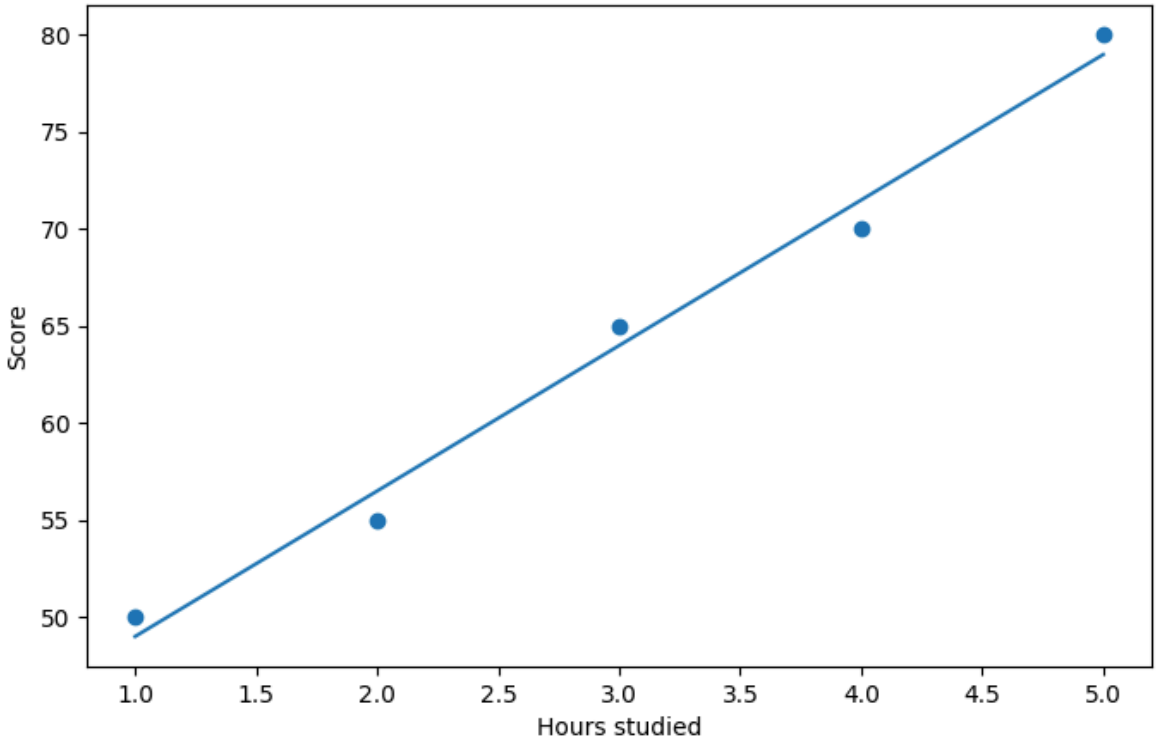
```
array([86.5])
```

   because predictions are always returned as a sequence (even for a single input).
9. Interpretation: for `x = 6` hours, the model predicts a score of $\hat{y} = 86.5$.

---

# 12. Visualizing the Model (Optional)

```
import matplotlib.pyplot as plt

plt.scatter(hours, score)
plt.plot(hours, model.predict(hours))
plt.xlabel("Hours studied")
plt.ylabel("Score")
plt.savefig("grade.png")
```




The line represents the learned relationship between input X (`hours`) and output Y (`score`).

---

## 13. `PRACTICE` Fitting a linear model

Use the following data:

```
import numpy as np

temp = np.array([60, 65, 70, 75, 80]).reshape(-1, 1)
sales = np.array([30, 45, 50, 65, 80])
```

Tasks:

1. Fit a linear model
2. Predict the value at 85$^ˆ$F
3. Plot the data and fitted line
4. In one complete sentence at the end, explain what the fitted line tells us about the relationship between temperature and ice cream sales.

Use Gemini in Colab to complete these tasks:

- Create code cell and click on `generate`
- In the Gemini console, change to `Gemini 3 Flash`

### 13.1. Solution

- Sample prompt:

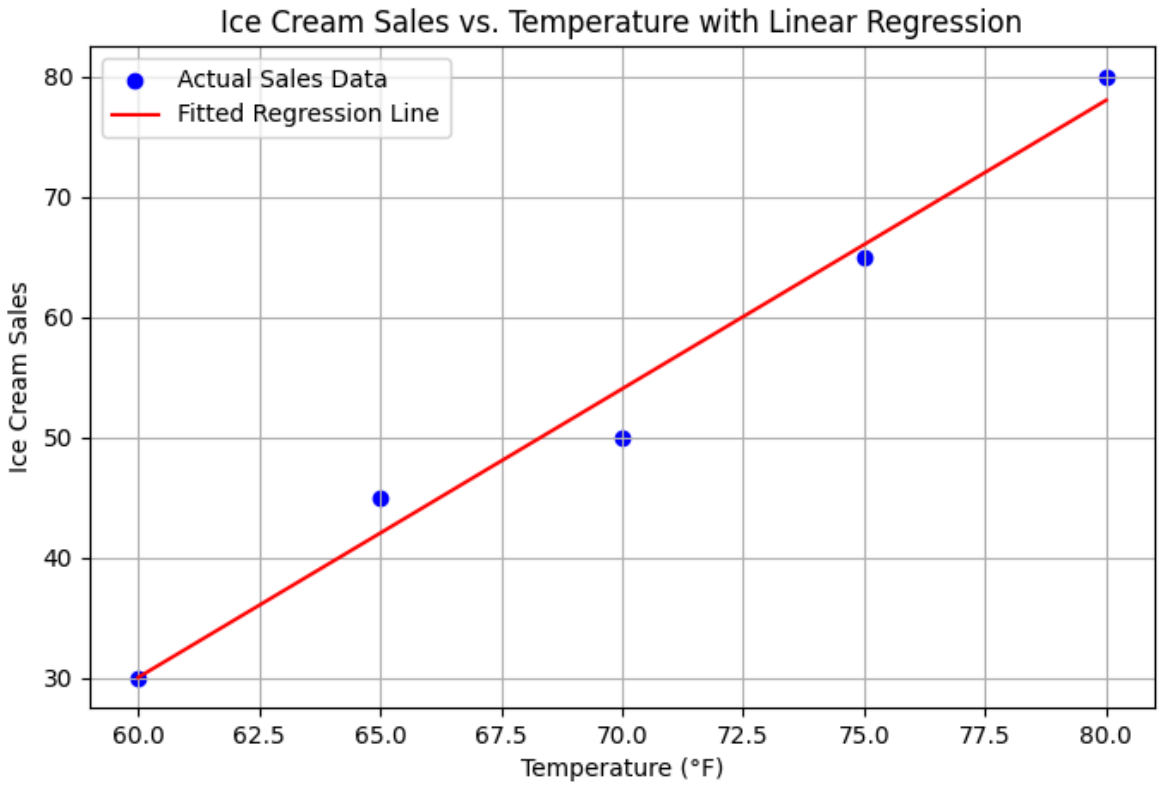
> "Can you complete the tasks for me? Comment amply in the code to help me understand what is going on."

- Response:

  > "I have generated the code to perform the tasks: fit a linear model, predict sales at 85°F, and visualize the data and the fitted line. The fitted line shows that as the temperature increases, ice cream sales also tend to increase, indicating a positive linear relationship between the two variables."

- Code with comments:

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# --- Task 1: Fit a linear model ---

# Re-using the data from the previous cell
# temp = np.array([60, 65, 70, 75, 80]).reshape(-1, 1)
# sales = np.array([30, 45, 50, 65, 80])

# Create a Linear Regression model instance
model_sales = LinearRegression()

# Fit the model to the 'temp' (independent variable) and 'sales' (dependent variable) data
# The .fit() method learns the relationship (slope and intercept) between the input and output
model_sales.fit(temp, sales)

# Print the learned parameters (slope and intercept) for understanding the model
print("Slope (coefficient):", model_sales.coef_[0])
print("Intercept:", model_sales.intercept_)

# --- Task 2: Predict the value at 85°F ---

# Create a numpy array for the temperature at which to make a prediction (85°F)
# It needs to be reshaped to a 2D array as expected by the predict method
temp_to_predict = np.array([[85]])

# Use the trained model to predict the sales for 85°F
predicted_sales_85 = model_sales.predict(temp_to_predict)
print(f"Predicted sales at 85°F: {predicted_sales_85[0]:.2f}")

# --- Task 3 & 4: Plot the data and fitted line ---

# Create a scatter plot of the original data points
plt.scatter(temp, sales, color='blue', label='Actual Sales Data')

# Plot the regression line
# To get the line, predict sales for the range of existing temperatures
plt.plot(temp, model_sales.predict(temp), color='red', label='Fitted Regression Line')

# Add labels and title for clarity
plt.xlabel("Temperature (°F)")
plt.ylabel("Ice Cream Sales")
plt.title("Ice Cream Sales vs. Temperature with Linear Regression")
plt.legend()
plt.grid(True)
plt.savefig("linear_regression2.png")
```




- Answer to the question:

  > The prediction at 85^ F represents the model's estimate of how many ice cream units will be sold at that temperature, based on the linear pattern learned from the data.

---

## 14. Key Points

- Machine learning learns patterns from data
- A model is a function that maps inputs (X) to outputs (y)
- Models are learned by minimizing error on training data
- Predictions are estimates, not exact values
- The workflow is iterative
- Models should generalize to new data
- Overfitting and underfitting affect model performance

---

## 15. Next Topic

In this lecture, we learned how to predict numerical values using a linear model.

> We moved from data → model → prediction.

Next, we will study a different kind of prediction:

> How can we decide between categories instead of predicting numbers?

We will introduce **classification**:

- Predicting labels such as pass/fail or spam/not spam

- Understanding how models make decisions
- Using similarity between data points (k-nearest neighbors)

  Instead of asking “What value?”, we will ask “Which class?”

We will also see how changing model parameters affects decisions, introducing the idea of model complexity.

## 16. Sources

All images: A. Geron, Hands-on Machine Learning with scikit-learn, Keras, and TensorFlow, 3e (O'Reilly, 2026).

Examples and code developed in collaboration with ChatGPT.

## Footnotes:

[1] 5V definition: Big Data are characterized by high volume, velocity, variety, veracity and value.

[2] Humans make the rules, machines execute them. This requires the ability to foresee everything that could possibly go wrong - which is impossible in a complex environment.

[3] This is not a reversal of the direction but about where the rules come from.

[4] After training, the model behaves as if it had rules, which however were never written by a human. This is called the **inference** phase, which is preceded by a **training** phase: `model=train(emails,labels)` where `labels` label emails spam/not spam.

[5] Example Google Photos: Once you upload all family photos, it automatically recognizes that A shows up in photos 1, 5, 11 while B shows up in 2, 5, 7 (unsupervised clustering). Label A and B and the service recognizes A and B in all photos.

[6] The model learns by predicting missing parts of the input, e.g. reconstructing a masked region of an image, using the surrounding context as its own supervision.

[7] This is exactly what pseudocode is: natural language without the constraints of syntactical rules. What used to be helpful in the past could in the future well become the standard for programming, cp.

Author: Marcus Birkenkrahe
Created: 2026-03-27 Fri 06:54